KUCA · 안드로이드 자동 배포

GitHub Actions + Firebase App Distribution 설정 가이드

한 번만 하는 준비 · 테스터 설치 · 평소 사용법

팀 저장소에 코드가 합쳐지면 GitHub 가 APK 를 빌드하고, Firebase 가 팀원·테스터 폰에 "새 버전" 알림을 보냅니다. 사람이 Unity 에서 빌드하고 파일을 돌릴 필요가 없습니다.

```
main 또는 feature/android-support 에 합쳐짐 (My project/ 아래가 바뀐 경우만)
  → GitHub Actions: Unity 6000.6.4f1 로 안드로이드 APK 빌드 (20~40분)
  → Firebase App Distribution 에 올림 → 테스터 그룹 kuca-team 에 알림
  → 테스터: Firebase App Tester 앱에서 "업데이트" 누르기
```

- 워크플로 파일: `.github/workflows/android-firebase.yml`
- 서버 쪽 변경(쪽지·AI 추천·날씨)은 이것과 상관없이 바로 반영됩니다. 이 배포는 앱 쪽(화면·게임 규칙·3D 모델) 변경용입니다.
- iOS 는 이 자동화에 포함되지 않습니다 (Mac 빌드 필요).

---

1. 한 번만 하는 준비 — 누가, 무엇을

순서	할 일	하는 사람	결과물 (→ Secrets 이름)
①	Firebase 프로젝트 만들고 안드로이드 앱 등록	Firebase 담당	앱 ID → `FIREBASE_APP_ID`
②	테스터 그룹 `kuca-team` 만들고 팀원 메일 추가	Firebase 담당	(그룹 별칭)
③	서비스 계정 키(JSON) 만들기	Firebase 담당	→ `FIREBASE_SERVICE_ACCOUNT`
④	서명 키(keystore) 만들기	안드로이드 담당 (B)	→ `ANDROID_KEYSTORE_BASE64`, `ANDROID_KEYSTORE_PASS`, `ANDROID_KEYALIAS_NAME`, `ANDROID_KEYALIAS_PASS`
⑤	Unity 라이선스 파일 꺼내기	Unity 계정 주인	→ `UNITY_LICENSE`, `UNITY_EMAIL`, `UNITY_PASSWORD`
⑥	위 9개를 팀 저장소 Secrets 에 등록	저장소 주인 (mungim5556-ui)	—

결정

- Firebase 프로젝트를 누구 구글 계정으로 만들지 (나머지 둘은 "사용자 및 권한"에 추가) 결정:
- 서명 키 원본 파일·비밀번호 보관 위치와 담당 (저장소에는 절대 올리지 않음) 결정:
- Unity 라이선스를 누구 계정으로 쓸지 (팀 공용 계정 권장) 결정:

① Firebase 프로젝트와 안드로이드 앱

1. console.firebase.google.com → 프로젝트 추가 → 이름 `KUCA` (Google 애널리틱스는 꺼도 됨)
2. 프로젝트 개요 → Android 앱 추가 - Android 패키지 이름: `com.mungim.kuca` (Unity Player Settings 의 Package Name 과 정확히 같아야 함) - `google-services.json` 다운로드·SDK 추가 단계는 건너뜁니다 (App Distribution 만 쓰면 필요 없음)
3. 프로젝트 설정(톱니바퀴) → 일반 → 내 앱 → 앱 ID 복사 (`1:123456789:android:abc...` 모양) → `FIREBASE_APP_ID`

② 테스터 그룹

1. 왼쪽 메뉴 App Distribution → 시작하기
2. 테스터 및 그룹 탭 → 그룹 추가 → 이름 아무거나, 별칭(alias) 을 `kuca-team` 으로 - 별칭을 바꾸면 워크플로 파일의 `TESTER_GROUPS` 도 같이 바꿉니다
3. 팀원 3명 메일 추가. 나중에 실사용 테스터(10~20명)는 별도 그룹(예: `kuca-pilot`)으로 만들고 `TESTER_GROUPS: kuca-team,kuca-pilot` 처럼 쉼표로 이어 씁니다

③ 서비스 계정 키 (GitHub 가 Firebase 에 올릴 때 쓰는 열쇠)

1. Firebase 프로젝트 설정 → 서비스 계정 탭 → "Google Cloud 에서 서비스 계정 관리" (또는 Google Cloud 콘솔 → 같은 프로젝트 → IAM 및 관리자 → 서비스 계정)
2. 서비스 계정 만들기 → 이름 `github-app-distribution`
3. 역할: Firebase App Distribution 관리자 (Firebase App Distribution Admin) 하나만
4. 만든 계정 → 키 탭 → 키 추가 → 새 키 만들기 → JSON → 파일이 내려받아짐
5. JSON 파일 내용 전체를 복사 → `FIREBASE_SERVICE_ACCOUNT`
6. 등록 뒤 JSON 파일은 지웁니다 (필요하면 키를 새로 만들면 됨)

④ 서명 키 (keystore)

왜 필요한가: 안드로이드는 같은 키로 서명된 APK 만 "업데이트"로 덮어씁니다. 키가 매번 바뀌면 테스터가 앱을 지우고 다시 깔아야 하고, 그때 폰에 저장된 진행 기록(`progress.json`)도 지워집니다.

방법 A — Unity 에서 (가장 쉬움) 1. Player Settings > Android > Publishing Settings > Keystore Manager → Create New → Anywhere 2. 파일 이름 `kuca.keystore` , 비밀번호, Alias `kuca` , Alias 비밀번호, 유효기간 50년 → Add Key

방법 B — 터미널 (`keytool` 은 JDK 에 있음. Unity 의 안드로이드 모듈 안 `OpenJDK/bin/` 에도 있음)

```
keytool -genkeypair -v -keystore kuca.keystore -alias kuca -keyalg RSA -keysize 2048 -validity 18250
```

Secrets 에 넣을 값

- `ANDROID_KEYSTORE_BASE64` : 파일을 base64 글자로 바꾼 것
- macOS: `base64 -i kuca.keystore | pbcopy` (클립보드로 복사됨)
- Windows PowerShell: `[Convert]::ToBase64String([IO.File]::ReadAllBytes("kuca.keystore")) | Set-Clipboard`
- `ANDROID_KEYSTORE_PASS` : keystore 비밀번호
- `ANDROID_KEYALIAS_NAME` : `kuca`
- `ANDROID_KEYALIAS_PASS` : alias 비밀번호

> ⚠️ `kuca.keystore` 파일과 비밀번호는 저장소에 올리지 말고, 위에서 정한 담당자가 따로 보관합니다 (예: 개인 구글 드라이브 비공개 폴더 + 비밀번호 관리자). 잃어버리면 모든 테스터가 앱을 지우고 다시 깔아야 합니다. 나중에 Google Play 에 올릴 때도 이 키를 씁니다.
>
> 내 PC 에서 직접 Build And Run 한 APK 도 같은 키로 서명하면(Player Settings > Publishing Settings 에 이 keystore 지정) 자동 배포판과 서로 덮어쓰기가 됩니다. 아니면 설치 전에 기존 앱을 지워야 합니다.

⑤ Unity 라이선스 (Personal 기준)

1. 그 계정으로 **Unity Hub** 에 로그인하고 Personal 라이선스를 활성화합니다 (보통 이미 돼 있음)
2. 라이선스 파일 `Unity_lic.ulf` 를 텍스트 편집기로 열어 내용 전체 복사 → `UNITY_LICENSE` - Windows: `C:\ProgramData\Unity\Unity_lic.ulf` - macOS: `/Library/Application Support/Unity/Unity_lic.ulf` - Linux: `~/.local/share/unity3d/Unity/Unity_lic.ulf`
3. 같은 계정의 메일·비밀번호 → `UNITY_EMAIL` , `UNITY_PASSWORD`

> 계정에 2단계 인증이 켜져 있으면 활성화가 실패할 수 있습니다. 실패하면 팀 공용 Unity 계정(2단계 인증 없음)을 따로 만들어 쓰는 것이 가장 간단합니다.

⑥ GitHub Secrets 등록 (저장소 주인만 가능)

`mungim5556-ui/KUCA` → **Settings → Secrets and variables → Actions → New repository secret** 에 아래 9개를 이름 그대로 등록합니다.

이름	값
`UNITY_LICENSE`	`Unity_lic.ulf` 내용 전체
`UNITY_EMAIL`	Unity 계정 메일
`UNITY_PASSWORD`	Unity 계정 비밀번호
`ANDROID_KEYSTORE_BASE64`	keystore 파일의 base64
`ANDROID_KEYSTORE_PASS`	keystore 비밀번호
`ANDROID_KEYALIAS_NAME`	`kuca`
`ANDROID_KEYALIAS_PASS`	alias 비밀번호
`FIREBASE_APP_ID`	`1:...:android:...`
`FIREBASE_SERVICE_ACCOUNT`	서비스 계정 JSON 내용 전체

Settings → Actions → General 에서 Actions 가 허용돼 있는지도 확인합니다.

> 비밀번호·키를 카카오톡·메일로 주고받지 마세요. 각자 자기 값을 저장소 주인 화면을 같이 보며 직접 입력하거나, 저장소 주인이 협업자에게 잠깐 관리자 권한을 주는 방법이 안전합니다.

---

2. 첫 실행 확인

1. 저장소 → **Actions** 탭 → "Android → Firebase" → **Run workflow** (브랜치 `feature/android-support`) → 메모 입력 → 실행
2. 첫 빌드는 캐시가 없어 30~40분 걸립니다. 다음부터는 `Library` 캐시로 빨라집니다
3. 성공하면 `kuca-team` 메일함에 초대 메일이 옵니다
4. 실패하면 맨 위 "Secrets 확인" 단계의 오류 메시지부터 봅니다 (빠진 Secrets 이름이 나옴). 그 밖의 오류는 아래 5장

---

3. 테스터(팀원 포함)가 할 일 — 처음 한 번

1. 안드로이드 폰에서 초대 메일 열기 → 시작하기 → 구글 계정으로 수락
2. 안내에 따라 **Firebase App Tester** 설치 (웹 클립 또는 앱)
3. App Tester 에서 KUCA → 다운로드 → "출처를 알 수 없는 앱 설치" 허용 → 설치
4. 이후 새 빌드가 올라오면 알림 → App Tester 에서 업데이트

---

4. 평소 사용법

하고 싶은 것	방법
새 기능을 폰에 보내기	PR 을 `main` 또는 `feature/android-support` 에 합치면 자동 (My project/ 아래가 바뀐 경우만)
지금 브랜치 상태로 한 번 더 보내기	Actions → Run workflow → 메모 입력
Firebase 없이 APK 만 받기	Actions → 해당 실행 → Artifacts 의 `KUCA····` (14일 보관)
버전 확인	앱 버전 `0.1.<실행 번호>` · App Tester 와 릴리스 메모에 커밋 번호
테스터 추가	Firebase → App Distribution → 테스터 및 그룹 → `kuca-team` 에 메일 추가

결정

- 자동 배포할 브랜치: 지금은 `main` + `feature/android-support`. 안드로이드 브랜치를 main 에 합친 뒤에는 `main` 만 남길지 결정:
- 실사용 테스트(10~20명) 그룹을 언제 만들고, 그 그룹에는 `main` 빌드만 보낼지 결정:

---

5. 자주 나는 문제

증상	원인 · 해결
"등록되지 않은 Secrets: …"	1장 ⑥ 표의 이름 그대로 등록했는지 (대소문자 포함)
Unity 활성화 실패 (License / activation)	`UNITY_LICENSE` 를 ulf 파일 전체로 넣었는지, 메일·비밀번호가 같은 계정인지, 2단계 인증 여부
`No space left on device`	워크플로의 "디스크 비우기" 단계 옵션을 `true` 로 늘림
`appdistribution:distribute` 403 / permission	서비스 계정 역할이 "Firebase App Distribution 관리자"인지, 같은 Firebase 프로젝트의 키인지
"앱이 설치되지 않음" / 서명 충돌	이전 APK 가 다른 키로 서명됨 → 한 번 지우고 다시 설치. 이후로는 같은 키라 업데이트됨
빌드가 너무 자주 돎	같은 브랜치에 새 커밋이 오면 이전 빌드는 자동 취소됨. 더 줄이려면 `on.push.branches` 에서 `feature/android-support` 를 뺌

---

6. 비용과 한도

- **Firebase App Distribution**: 무료
- **GitHub Actions**: 공개 저장소면 무료. 비공개 저장소면 계정 무료 사용량(월 2,000분) 안에서 — 빌드 1회 약 20~40분, 한 달 50회 남짓

- 빌드 시간 줄이기: `My project/` 아래가 바뀐 커밋만 빌드하고, 같은 브랜치의 이전 빌드는 취소함 (이미 설정됨)